

XX搜题 APK 脱壳重打包与去广告 调试报告

—— 360 加固脱壳、字节级 Dex 补丁、兼容性修复全流程

最终版本: V74 | 测试环境: 雷电模拟器 + 华为 P30 (鸿蒙 4.0)

一、项目背景与目标

1.1 项目背景

用户提供了一个经过 360 加固的 Android APK——"小包搜题"（包名：com.ixyzh.question）。该应用是一款题库搜题工具，支持扫描搜题、智能搜题、录屏搜题、便捷搜题等功能，支持导入 Excel 格式的自定义题库。

由于应用经过 360 加固，无法直接反编译修改。同时应用内置了大量广告（开屏广告、Banner 广告等），影响使用体验。用户明确要求"脱壳重打包重新签名"，拒绝任何不脱壳的"委屈方案"。

1.2 核心目标

1. 对 360 加固的 APK 进行脱壳，提取真实的 Dex 代码
2. 重打包并重新签名，生成可安装的 APK
3. 去除应用内所有广告（开屏广告、Banner 广告等）
4. 保证核心功能可用：题库导入、四个搜题功能（扫描/智能/录屏/便捷）
5. 解决兼容性问题：华为 P30（鸿蒙 4.0）闪退、微信 SDK 崩溃、存储权限申请等
6. 优化用户体验：存储权限自动申请、授权后即时生效

1.3 验收标准

- 用"信息安规最新版题库.xlsx"（285 题）导入成功、不报错
- 主界面"我的下载"中能看到导入的题库
- 四个搜题功能（扫描搜题、智能搜题、录屏搜题、便捷搜题）点击后能看到用户导入的题库
- 华为 P30（鸿蒙 4.0）上不闪退
- 无广告展示

二、整体技术思路

2.1 技术路线选择

针对 360 加固的 APK，有以下几种技术路线：

方案 A：使用 FART/Youpk 等脱壳框架，在定制 ROM 中运行 App 并 dump 出 Dex

优点：脱壳彻底，能拿到完整的 Dex

缺点：需要定制 ROM 或特定环境，配置复杂

方案 B：运行时内存 Dump，在 App 运行时从内存中提取解密后的 Dex

优点：不需要定制 ROM，普通模拟器即可

缺点：需要找到正确的 dump 时机，可能拿到不完整的 Dex

方案 C：Hook 脱壳，使用 Frida/Xposed 等框架 Hook 系统 API，在 Dex 加载时提取

优点：精准，能拿到原始 Dex

缺点：需要配置 Hook 环境，360 可能有反调试

本项目采用方案 B（运行时内存 Dump），在雷电模拟器中运行 App，通过 DexDump 类从内存中 dump 出解密后的 Dex。

2.2 整体流程

整个调试过程分为以下几个阶段：

阶段 1：脱壳 —— 从 360 壳内存中 dump 出真实 Dex (classes60_v2.dex)

阶段 2：构建基础壳 APK —— 用原版 APK 的 zip 骨架，替换 classes.dex 为脱壳后的 Dex

阶段 3：字节级 Dex 补丁 —— 用 smali/baksmali 工具修改 Dex 中的关键方法（去广告、修复功能）

阶段 4：重打包签名 —— 替换 Dex、zipalign 对齐、apksigner 签名

阶段 5：测试验证 —— 模拟器+真机双环境测试，迭代修复问题

阶段 6：兼容性优化 —— 修复华为 P30 闪退、微信崩溃、权限申请等

2.3 为什么选择字节级 Dex 补丁而非 apktool 回编译

传统的 APK 修改通常使用 apktool 反编译→修改 smali→回编译。但本项目选择了字节级 Dex 补丁（直接修改 Dex 文件，然后替换进 APK），原因如下：

1. 避免资源问题：apktool 回编译可能导致 resources.arsc、AndroidManifest.xml 等资源文件损坏，尤其是经过加固的 APK
2. 保留原始签名结构：只替换 classes.dex，其他文件保持原样，减少兼容性问题
3. 更快的迭代速度：每次修改只需要重新编译 smali→替换 Dex→签名，不需要完整回编译
4. 关键约束：resources.arsc 必须是 APK 中第一个文件且不压缩（ZIP_STORED），否则资源加载失败

三、工具清单与用途

3.1 脱壳与分析工具

工具名称	版本/路径	用途
DexDump 类	自定义 Java 类	从 360 壳内存中 dump 出解密后的 Dex 文件，运行时内存 dump
jadx-gui	1.5.0 (带 JRE)	APK 反编译查看，将 Dex 反编译为 Java 代码，分析业务逻辑
apktool	2.12.1 / 3.0.3	APK 反编译/回编译，提取 resources.arsc、AndroidManifest.xml、smali 代码、assets 资源
baksmali.jar	ApkToolBox 内置	Dex 反汇编为 smali 代码，用于分析和修改 Dex 字节码
smali.jar	ApkToolBox 内置	smali 汇编为 Dex 文件，将修改后的 smali 代码重新编译为 Dex
dexdump.exe	build-tools-r30	Dex 反汇编查看，输出 Dex 的完整字节码和方法信息，用于精确分析方法地址和指令
dex2jar	2.1 (ApkToolBox 内置)	Dex 转 Jar 工具集，包含 dex2jar、jar2dex 等 30+ 个子工具，可将 Dex 转为 jar 后用 jd-gui 查看
jd-gui.exe	ApkToolBox 内置	Java 反编译查看器，将 jar/class 文件反编译为 Java 源代码，配合 dex2jar 使用
ApkToolBox Studio	v1.6.4	APK 工具集合 GUI，集成了 apktool、smali、baksmali、dex2jar、jd-gui、aapt、zipalign、apksigner 等工具

frida-server	17.17.0 / 16.7.19 (android-x86_64)	动态插桩工具服务端，运行在 Android 设备上，配合 PC 端 frida 客户端进行 Hook 调试、运行时分析、方法调用追踪、绕过检测、修改运行时数据
aapt.exe	build-tools-r30	Android Asset Packaging Tool，资源打包工具，用于查看、打包、解压 APK 资源
aapt2.exe	build-tools-r30	aapt 的升级版，支持增量资源编译，用于 Android Studio 构建 APK
d8.jar	build-tools-r30	Android D8 Dex 编译器，将 Java 字节码(.class)编译为 Dex 格式，支持 desugaring
dx.jar	build-tools-r30	旧版 Dex 编译器，将 Java 字节码编译为 Dex 格式，已被 d8 取代但仍兼容

3.2 打包与签名工具

工具名称	路径	用途
zipalign.exe	build-tools-r30	APK 对齐优化，确保 APK 内文件 4 字节对齐，优化运行时内存占用
apksigner.jar	build-tools-r30	APK 签名工具，支持 v1(JAR 签名)+v2(APK Signature Scheme v2)双方案签名
keytool.exe	JRE 内置	Java 密钥工具，用于生成和管理密钥库(keystore)，创建签名用的密钥对
test.keystore	工作目录生成	签名密钥库文件，密码 123456，别名 test，用于 APK 签名
Java 运行环境	jadx-gui 内置 JRE	运行 smali/apksigner 等

		Java 工具的运行环境
--	--	--------------

3.3 调试与测试工具

工具名称	路径/型号	用途
adb.exe	D:\工具软件\雷电模拟器9\LDPlayer14\adb.exe	Android 调试桥，安装 APK、查看日志、截图、模拟点击
雷电模拟器	LDPlayer14 (emulator-5554)	Android 模拟器，用于初步测试
华为 P30	ELE-AL00 (鸿蒙 4.0/Android 10)	真机测试，验证兼容性
logcat	adb logcat	查看 Android 系统日志，定位崩溃和异常
uiautomator dump	adb shell uiautomator dump	导出当前界面 UI 层级，分析控件坐标
sqlite3	命令行工具	查看 App 数据库 (question.db)，验证题库导入

3.4 开发与脚本工具

- Python 3.8: 编写补丁脚本 (patch_v6.py、patch_v7.py、fix_*.py 等)，自动化 Dex 修改和 APK 构建
- 自定义辅助脚本: ddgrep.py (检索 dexdump 输出)、ui_nodes.py (解析 UI 层级)、build_v7.py (自动化构建) 等
- PowerShell: Windows 命令行，执行 adb 命令、文件操作、自动化脚本

四、调试过程详解

4.1 阶段 1：脱壳

360 加固的原理：将原始 Dex 加密后存放在 APK 的 assets 或 lib 目录中，运行时由壳代码（libjiagu.so 等）解密并加载到内存中。原始的 classes.dex 只是壳代码，不包含真实业务逻辑。

脱壳步骤：

1. 在雷电模拟器中安装原版 APK 并启动
2. 使用 DexDump 类遍历内存中的 Dex 文件，找到包含真实业务逻辑的 Dex（classes60.dex）
3. 将内存中的 Dex dump 出来，保存为 classes60_v2.dex（约 9MB）
4. 用 jadx-gui 验证 dump 出的 Dex，确认包含 LoadQuestionActivity、MyDownloadActivity 等真实业务类

关键发现：dump 出的 Dex 中，LoadQuestionActivity（导入题库页面）、MyDownloadActivity（我的下载页面）、SplashActivity（开屏页面）等关键类都存在，可以直接修改。

4.2 阶段 2：构建基础壳 APK

构建步骤：

1. 复制原版 APK 作为基础骨架（deshell_v5.apk）
2. 用 zipfile 工具打开 APK，删除原始的 classes.dex
3. 将脱壳后的 classes60_v2.dex 重命名为 classes.dex，添加到 APK 中
4. 关键约束：resources.arsc 必须是 APK 中第一个文件且使用 ZIP_STORED（不压缩），否则 Android 资源加载失败
5. zipalign 对齐 → apksigner 签名 → 安装测试

第一次测试结果：App 能启动，但导入题库后"我的下载"中看不到题库，且有广告展示。需要进一步修改 Dex 代码。

4.3 阶段 3：去广告与功能修复（字节级 Dex 补丁）

4.3.1 补丁链概述

采用迭代式补丁链，每个版本在之前版本的基础上继续修改：

- v6：重建 LoadQuestionActivity.onCreate 和 \$1.handleMessage（去广告分支）
- v7：重建 MyDownloadActivity.onCreate/onResume/onActivityResult（去 Banner 广告、直刷列表）+ 给 LoadQuestionActivity\$7.event 插入 setCurver(1)
- v66 及以后：基于 apktool 反编译的完整 smali 代码进行修改，添加权限申请、兼容性修复等

4.3.2 题库导入功能修复

问题现象：导入题库后，questionrecord 表有 285 条记录，但 tikurecord 表为 0 条，"我的下载"中看不到导入的题库。

根因分析：

1. MyDownloadActivity 的列表数据源是 TikuManager.getDownloadTikuList()
2. 该方法使用 SQL 条件 "curver > 0" 过滤题库记录
3. 导入弹窗确认后，创建 TikuRecord 时没有设置 curver 字段（默认为 0）
4. 因此导入的题库记录 curver=0，被过滤条件排除，不出现在"我的下载"中

修复方案：在 LoadQuestionActivity\$7.event 方法（导入确认按钮的事件回调）中，在 setTid 之后、save 之前，插入 setCurver(1)调用。

修改前（\$7.event 方法片段）：

```
invoke-virtual {v0, v1}, Lcom/ixyzh/question/bean/TikuRecord;->setTid(J)V
...
invoke-virtual {v0}, Lcom/ixyzh/question/bean/TikuRecord;->save()Z
```

修改后：

```
invoke-virtual {v0, v1}, Lcom/ixyzh/question/bean/TikuRecord;->setTid(J)V
```

```
const/4 v1, 0x1
invoke-virtual {v0, v1}, Lcom/ixyzh/question/bean/TikuRecord; -> setCurver(I)V
...
invoke-virtual {v0}, Lcom/ixyzh/question/bean/TikuRecord; -> save()Z
```

验证结果：导入 285 条题后，tikurecord 表有 1 条记录且 curver=1，"我的下载"中能看到导入的题库。

4.3.3 去广告修改

1. SplashActivity（开屏页）：移除开屏广告加载逻辑，直接延迟跳转主界面
2. MyDownloadActivity（我的下载页）：移除 Banner 广告加载，直刷列表数据
3. LoadQuestionActivity\$1.handleMessage：去除广告相关分支，直接处理导入结果
4. 保留 mForceGoMain 机制：onStop 时设置 mForceGoMain=true，onResume 时如果为 true 则跳转主界面，确保从后台返回时能正常进入

4.4 阶段 4：四个搜题功能验证

逐一验证四个搜题功能：

1. 扫描搜题（OcrScannerActivity）：点击后进入扫描界面，能选择导入的题库
2. 智能搜题（SmartGuideActivity）：点击后进入智能搜题界面，能选择导入的题库
3. 录屏搜题：需要悬浮窗权限，授权后能显示悬浮窗，从题库中取题展示
4. 便捷搜题：点击后能选择导入的题库

验证标准：每个功能点击后，在题库选择列表中能看到用户导入的题库（curver>0 的记录）。

4.5 阶段 5：兼容性修复

在华为 P30（鸿蒙 4.0，基于 Android 10）上测试时遇到多个兼容性问题，详见第五章"关键代码修复详解"。

4.6 阶段 6：权限体验优化

1. 存储权限自动申请：SplashActivity 启动时检测
READ_EXTERNAL_STORAGE/WRITE_EXTERNAL_STORAGE，没有则弹出系统授权框

2. 授权后即时生效: onRequestPermissionsResult 中调用 recreate()重建 Activity, 权限即时生效, 无需手动重启 App

3. Android 11+所有文件访问权限: 自动跳转到系统设置页, onResume 重新检测, 授权后即时生效

五、关键代码修复详解

5.1 修复一：华为 P30 闪退 (NoSuchMethodError)

问题现象：V68 在华为 P30 (鸿蒙 4.0) 上启动即闪退，logcat 显示：

```
java.lang.NoSuchMethodError: No static method isExternalStorageManager()Z
    in class Landroid/os/Environment;
```

根因分析：

Environment.isExternalStorageManager() 是 Android 11 (API 30) 才新增的方法，用于检测"所有文件访问"权限。华为 P30 的鸿蒙 4.0 基于 Android 10 (API 29)，该方法不存在，调用时抛出 NoSuchMethodError。

修复方案：在调用 isExternalStorageManager()之前，先检查 SDK 版本，只有 API ≥ 30 才调用。

修改前 (SplashActivity.onCreate)：

```
invoke-static {}, Landroid/os/Environment;->isExternalStorageManager()Z
move-result v0
if-nez v0, :has_storage_perm
# 跳转设置页...
```

修改后：

```
sget v0, Landroid/os/Build$VERSION;->SDK_INT:I
const/16 v1, 0x1e    # 30
if-lt v0, v1, :has_storage_perm    # SDK < 30, 跳过
invoke-static {}, Landroid/os/Environment;->isExternalStorageManager()Z
move-result v0
if-nez v0, :has_storage_perm
# 跳转设置页...
```

同样的修复也应用于 onResume 方法，确保从设置页返回时也能正确检测。

5.2 修复二：微信 SDK 崩溃

问题现象：App 启动时崩溃，logcat 显示微信 SDK 相关的异常。

根因分析：App 在 Application.onCreate 中调用 regToWx()注册微信 SDK，但设备上没有安装微信或微信 SDK 初始化失败，导致崩溃。

修复方案：将 regToWx()方法改为直接 return-void，跳过微信注册。

修改前:

```
.method private regToWx()V
    .registers 3
    # 微信 SDK 初始化代码...
    invoke-virtual {v0, v1, v2}, Lcom/tencent/mm/opensdk/openapi/WXAPIFactory;-
>createWXAPI(...)
    ...
.end method
```

修改后:

```
.method private regToWx()V
    .registers 1
    return-void
.end method
```

5.3 修复三: VerifyError 寄存器类型混淆 (long 参数)

问题现象: V10 首次安装后点导入崩溃, logcat 显示:

```
java.lang.VerifyError: register v1 has type Imprecise Constant: 32767
    but expected Long (Low Half)
```

根因分析:

在调用 `sendEmptyMessageDelayed(IJ)` 方法时, 第二个参数是 `long` 类型 (64 位), 需要使用一对寄存器 (`v1,v2`)。但代码中用 `const/16 v1` 初始化了一个 `int` 值 (32 位), ART 验证器期望 `v1` 是 `Long` 的低半部分, 实际却是 `int` 常量, 导致 `VerifyError`。

修复方案: 使用 `const-wide/16` 指令初始化 `long` 类型的寄存器对。

修改前:

```
const/16 v1, 0x7fff    # 错误: 用 int 指令初始化 long 参数
    invoke-virtual {v0, v1, v2}, Landroid/os/Handler;-
>sendEmptyMessageDelayed(IJ)Z
```

修改后:

```
const-wide/16 v1, 0x7fff    # 正确: 用 wide 指令初始化 long 寄存器对
    invoke-virtual {v0, v1, v2}, Landroid/os/Handler;-
>sendEmptyMessageDelayed(IJ)Z
```

5.4 修复四: 存储权限自动申请

问题现象: 用户需要手动在设置中开启存储权限, 否则导入题库失败。

修复方案: 在 `SplashActivity.onCreate` 中添加权限检测和申请逻辑。

```

# 检查存储权限
const-string v0, "android.permission.READ_EXTERNAL_STORAGE"
invoke-virtual {p0, v0}, Landroid/content/Context;
->checkSelfPermission(Ljava/lang/String;)I
move-result v0
const/4 v1, 0x0      # PERMISSION_GRANTED
if-eq v0, v1, :has_runtime_perm

# 没有权限, 申请
const/4 v0, 0x2
new-array v0, v0, [Ljava/lang/String;
const-string v1, "android.permission.READ_EXTERNAL_STORAGE"
const/4 v2, 0x0
aput-object v1, v0, v2
const-string v1, "android.permission.WRITE_EXTERNAL_STORAGE"
const/4 v2, 0x1
aput-object v1, v0, v2
const/16 v1, 0x64     # requestCode
invoke-virtual {p0, v0, v1}, Landroid/app/Activity;
->requestPermissions([Ljava/lang/String;)V
return-void          # 申请后返回, 等待用户授权

:has_runtime_perm
# 继续执行原来的逻辑...

```

踩坑记录:

1. 最初误用 `ActivityCompat.requestPermissions` (静态方法), 用 `invoke-virtual` 调用导致 `VerifyError`。应直接调用 `Activity` 的 `requestPermissions` (虚方法)。
2. `const/4` 只能加载-8 到 7 的整数, `requestCode=100` 需要用 `const/16`。

5.5 修复五: 权限授权后即时生效 (recreate 方案)

问题现象: 用户在权限申请框中点击"允许"后, App 继续运行但权限不生效, 需要手动重启 App 才能导入题库。

根因分析:

最初的实现中, 申请权限后代码继续执行 (没有 `return`), 不管用户是否授权都进入主界面。用户授权后返回 App 时, `SplashActivity` 已经执行完毕, 权限状态没有被重新检测。

修复方案:

1. 申请权限后立即 `return-void`, 不继续执行原来的逻辑

2. 重写 onRequestPermissionsResult 方法，用户授权后调用 recreate()重建 Activity

3. recreate()会导致 SplashActivity 被销毁并重新创建，onCreate 被重新调用，此时检测权限已授权，继续执行原来的逻辑

```
.method public onRequestPermissionsResult(I[Ljava/lang/String;[I)V
    .registers 6

    const/16 v0, 0x64    # requestCode
    if-ne p1, v0, :cond_0

    array-length v0, p3
    if-eqz v0, :cond_0

    const/4 v0, 0x0
    aget v0, p3, v0
    if-nez v0, :cond_0    # 不是 PERMISSION_GRANTED

    # 用户授权了，重建 Activity
    invoke-virtual {p0}, Landroid/app/Activity;->recreate()V
    return-void

:cond_0
    invoke-super {p0, p1, p2, p3}, Landroidx/appcompat/app/AppCompatActivity;-
    >onRequestPermissionsResult(I[Ljava/lang/String;[I)V
    return-void
.end method
```

踩坑记录：.registers 必须足够（至少 6 个），否则 v0 被当成参数寄存器（this 引用），导致"tried to get class from non-reference register"的 VerifyError。

六、踩坑记录

6.1 脱壳阶段

1. 360 加固的 Dex 在内存中是动态加载的，需要在 App 完全启动后再 dump，否则可能拿到不完整的 Dex
2. dump 出的 Dex 可能有多个 (classes.dex、classes2.dex...)，需要找到包含真实业务逻辑的那个 (本项目是 classes60.dex)
3. 用 jadx-gui 验证 dump 出的 Dex 时，如果出现解析错误，可能是 dump 不完整，需要重新 dump

6.2 打包阶段

1. resources.arsc 必须是 APK 中第一个文件且使用 ZIP_STORED (不压缩)，否则 Android 资源加载失败，App 闪退
2. META-INF 目录 (签名文件) 必须删除，否则重新签名后验证失败
3. zipalign 必须在签名之前执行，否则 v2 签名会失效
4. apksigner 签名需要同时使用 v1 和 v2 方案，确保 Android 7.0 以下和以上都能安装

6.3 smali 修改阶段

1. .registers 数量必须足够，否则会出现 VerifyError (寄存器类型混淆)。添加新代码时要重新计算寄存器数量
2. const/4 只能加载 -8 到 7 的整数，const/16 加载 -32768 到 32767，const/32 加载任意 32 位整数。用错会导致编译失败或 VerifyError
3. long 和 double 类型需要使用一对寄存器 (v0,v1)，必须用 const-wide 指令初始化，不能用 const/4 或 const/16
4. 静态方法用 invoke-static 调用，虚方法用 invoke-virtual 调用，接口方法用 invoke-interface 调用。用错会导致 VerifyError

5. 修改方法后，方法的字节码长度可能变化，需要确保跳转标签（:cond_0、:goto_0 等）的偏移正确。smali 汇编器会自动计算偏移，但如果标签名称冲突会导致错误
6. 新增方法时，要确保方法签名（参数和返回值类型）正确，否则调用时会出现 `NoSuchMethodError`

6.4 调试阶段

1. adb 设备离线：USB 线接触不良或模拟器网络问题，需要 `adb kill-server && adb start-server` 重新连接
2. 安装失败（`INSTALL_FAILED_UPDATE_INCOMPATIBLE`）：签名不同无法覆盖安装，需要先卸载旧版本
3. 华为 P30 安装时需要在手机上手动确认安装对话框，否则会超时
4. logcat 缓冲区有限，崩溃后要及时查看，否则日志会被覆盖。使用 `adb logcat -d -t 100` 查看最近 100 行
5. 鸿蒙系统的权限管理与原生 Android 有差异，部分权限需要用户手动在设置中开启

6.5 兼容性阶段

1. Android 10（API 29）没有 `isExternalStorageManager()` 方法，必须加 SDK 版本判断
2. 鸿蒙 4.0 基于 Android 10，但有自己的权限管理机制，存储权限申请框的样式与原生 Android 不同
3. 没有安装微信的设备上，微信 SDK 初始化会崩溃，需要跳过注册
4. `recreate()` 方法在某些设备上可能导致闪烁，但能确保权限即时生效，是可接受的权衡

七、技术亮点

7.1 纯字节级 Dex 补丁

本项目没有使用 apktool 完整回编译，而是采用字节级 Dex 补丁方案：

- 只修改 classes.dex，其他文件（resources.arsc、AndroidManifest.xml、assets、lib 等）保持原样
- 避免了 apktool 回编译导致的资源损坏、Manifest 解析错误等问题
- 迭代速度快，每次修改只需要重新编译 smali→替换 Dex→签名，整个过程不到 1 分钟
- 关键约束：resources.arsc 必须是 APK 第一个文件且不压缩，这是通过 Python zipfile 精确控制写入顺序实现的

7.2 存储权限自动申请+即时生效

用户体验优化的核心亮点：

- 启动时自动检测存储权限，没有则弹出系统授权框（不需要用户手动去设置）
- 用户授权后通过 recreate()自动重建 Activity，权限即时生效（不需要手动重启 App）
- 申请权限后 return-void，确保不会在未授权状态下继续执行
- onRequestPermissionsResult 中精确判断 requestCode 和授权结果，只在用户授权时才 recreate

7.3 多版本迭代修复

从 v6 到 v74，经历了数十个版本的迭代：

- v6-v7：基础脱壳和去广告
- v10：修复 VerifyError（long 参数寄存器混淆）
- v66：基于 apktool 完整反编译，添加微信崩溃修复
- v68：修复华为 P30 闪退（SDK 版本判断）

- v70: 添加 onResume 权限检测 (所有文件访问权限即时生效)
- v71-v73: 存储权限自动申请 (修复 invoke-virtual、.registers 等问题)
- v74: 最终版本, 权限授权后即时生效 (recreate 方案)

7.4 模拟器+真机双环境验证

- 雷电模拟器 (Android 7.1+) : 用于初步测试和快速迭代
- 华为 P30 (鸿蒙 4.0/Android 10) : 用于真机兼容性验证
- 双环境测试确保了 App 在不同 Android 版本和厂商定制系统上的稳定性

7.5 数据库验证

通过 sqlite3 直接查看 App 数据库 (question.db) , 验证题库导入结果:

- questionrecord 表: 验证 285 条题目是否正确导入
- tikurecord 表: 验证题库记录是否创建、curver 是否为 1
- 这种"白盒验证"方式比单纯看 UI 更准确, 能快速定位问题根因

八、最终版本说明

8.1 V74 功能清单

功能	状态	说明
脱壳重打包	✓	360 加固脱壳，重新签名
去广告	✓	开屏广告、Banner 广告全部去除
华为 P30 不闪退	✓	SDK 版本判断，避免调用 Android 11+ 方法
微信 SDK 崩溃修复	✓	regToWx 直接 return-void
存储权限自动申请	✓	启动时检测，没有则弹系统授权框
权限授权后即时生效	✓	recreate()重建 Activity，无需手动重启
Android 11+所有文件访问	✓	半自动跳转设置页，onResume 重新检测
题库导入 (285 条)	✓	setCurver(1)修复，"我的下载"可见
扫描搜题	✓	能选择导入的题库
智能搜题	✓	能选择导入的题库
录屏搜题	✓	悬浮窗权限授权后可用
便捷搜题	✓	能选择导入的题库

8.2 签名信息

- 密钥库：test.keystore
- 密码：123456
- 别名 (alias)：test
- 签名方案：v1 (JAR 签名) + v2 (APK Signature Scheme v2)
- SHA-256 指纹：
1822fce2eb3065fa35c68892fe6d1ee4899feec5c719295d8b8248f22bbaa4b

8.3 兼容性

- 最低支持: Android 10 (API 29) / 鸿蒙 4.0
- 推荐支持: Android 11+ (API 30+) , 支持"所有文件访问"权限自动申请
- 测试设备: 雷电模拟器 (Android 7.1+) 、 华为 P30 (鸿蒙 4.0/Android 10)

8.4 文件归档说明

所有相关文件已归档到 D:\桌面\小包搜题_去广告_项目归档\ 目录:

- 01_文档/: 本调试报告 (Word 文档)
- 02_最终 APK/: 小包搜题_去广告_V74.apk (最终交付版本)
- 03_历史版本/: v66、v68、v70、v71、v72、v73 等历史版本
- 04_源码与脚本/: 所有 Python 补丁脚本、smali 代码
- 05_签名密钥/: test.keystore
- 06_工具软件/: adb、smali.jar、zipalign、apksigner 等工具
- 07_原始文件/: 原版 APK、题库文件
- 08_dex 文件/: 脱壳后的各个版本 Dex 文件

—— 报告完 ——